

SEMANA 3 — DÍA 2

Visualización básica de datos

 **Objetivo del día:** Aprender a generar visualizaciones simples, correctas y útiles para explorar los datos limpios del Script 1, entendiendo qué hace cada tipo de gráfica, cómo personalizarla y cómo evitar errores comunes.

TEORÍA

1. ¿Qué es una gráfica?

Una gráfica es una **representación visual de datos**. Su propósito es ayudarte a **ver patrones** que no son evidentes en una tabla o lista.

Una buena gráfica debe:

- Mostrar la información de forma clara
- No distorsionar los datos
- Tener etiquetas que expliquen qué se está viendo
- Tener un título descriptivo
- Usar colores que no confundan
- Ser simple (sin elementos innecesarios)

Una gráfica **NO** debe:

- Exagerar diferencias
- Ocultar información
- Usar efectos 3D que distorsionan
- Usar escalas engañosas
- Sugerir causalidad cuando solo hay correlación

2. ¿Qué debe tener una buena gráfica?

Toda gráfica debe incluir:

✓ **Título** - Explica qué estás mostrando.

```
plt.title("Distribución de alturas")
```

✓ **Etiquetas de ejes** - Indican qué representa cada eje.

```
plt.xlabel("Altura (cm)")  
plt.ylabel("Frecuencia")
```

✓ **Escalas claras** - Evitar ejes truncados sin aviso.

✓ **Colores adecuados** - Evitar colores muy saturados o que dificulten la lectura.

```
plt.hist(lista, color="skyblue")
```

✓ **Leyenda (si aplica)**

```
plt.legend(["Alturas"])
```

✓ **Guardar la gráfica** - Para usarla en el PIA.

```
plt.savefig("figures/hist_alturas.png")
```

3. Tipos de gráficas en Matplotlib (explicadas a detalle)

A continuación tienes **qué hace cada gráfica, cuándo usarla, ejemplos con listas, y cómo personalizarla**.

3.1 Histograma — Distribución de una variable

✓ **¿Qué muestra?** Cómo se distribuyen los valores de una variable numérica.

✓ **¿Cuándo usarlo?**

- Para ver si los datos son simétricos
- Para detectar valores atípicos
- Para ver si hay varios “picos”

✓ **Ejemplo:**

```
import matplotlib.pyplot as plt

alturas = [10,12,12,13,15,18,20,21,22,22,23]

plt.hist(alturas, bins=8, color="purple", edgecolor="black")
plt.title("Distribución de alturas")
plt.xlabel("Altura")
plt.ylabel("Frecuencia")
plt.grid(axis="y", alpha=0.3)
plt.savefig("figures/hist_alturas.png")
plt.show()
```

3.2 Gráfica de barras — Comparaciones

✓ **¿Qué muestra?** Comparaciones entre categorías.

✓ **¿Cuándo usarla?**

- Comparar tipos
- Comparar frecuencias
- Comparar valores por categoría

✓ **Ejemplo:**

```
import matplotlib.pyplot as plt
categorias = ["A", "B", "C"]
valores = [10, 15, 7]

plt.bar(categorias, valores, color=["#4C72B0", "#55A868", "#C44E52"])
plt.title("Comparación por categoría")
plt.xlabel("Categoría")
plt.ylabel("Valor")
plt.savefig("figures/barras.png")
plt.show()
```

3.3 Gráfica de líneas — Tendencias

✓ **¿Qué muestra?** Cómo cambia una variable a lo largo del tiempo o de un índice.

✓ **¿Cuándo usarla?**

- Series de tiempo
- Evolución
- Tendencias

✓ **Ejemplo:**

```
import matplotlib.pyplot as plt
dias = [1,2,3,4,5]
temperaturas = [20,22,21,23,24]

plt.plot(dias, temperaturas, marker="o", linestyle="--", color="orange")
plt.title("Temperatura por día")
plt.xlabel("Día")
plt.ylabel("Temperatura")
```

```
plt.savefig("figures/linea_temperatura.png")
plt.show()
```

3.4 Gráfica de dispersión — Relación entre dos variables

✓ **¿Qué muestra?** Si dos variables están relacionadas.

✓ **¿Cuándo usarla?**

- Para ver correlaciones
- Para detectar patrones
- Para ver si hay agrupaciones

✓ **Ejemplo:**

```
import matplotlib.pyplot as plt
pesos = [50,55,60,65,70,75]
alturas = [150,155,160,165,170,175]

plt.scatter(pesos, alturas, color="green")
plt.title("Relación peso-altura")
plt.xlabel("Peso")
plt.ylabel("Altura")
plt.savefig("figures/dispersion_peso_altura.png")
plt.show()
```

3.5 Gráfica de pastel — Proporciones

✓ **¿Qué muestra?** Cómo se divide un total entre categorías.

✓ **¿Cuándo usarla?***

- Cuando quieres ver proporciones
- Cuando hay pocas categorías (máximo 5)

✗ **¿Cuándo NO usarla?**

- Cuando las diferencias son pequeñas
- Cuando hay muchas categorías
- Cuando necesitas precisión

✓ **Ejemplo:**

```
import matplotlib.pyplot as plt
categorias = ["Fuego", "Agua", "Planta"]
porcentajes = [40, 35, 25]

plt.pie(porcentajes, labels=categorias, autopct="%1.1f%%",
        colors=["#FF5733", "#33C1FF", "#75FF33"])
plt.title("Distribución de tipos")
plt.savefig("figures/pastel_tipos.png")
plt.show()
```

4. Personalización general de gráficas

Puedes personalizar:

✓ **Tamaño de la figura**

```
plt.figure(figsize=(8,5))
```

✓ **Colores**

```
color="skyblue"
```

✓ **Líneas y marcadores**

```
linestyle="--"
marker="o"
```

✓ Transparencia

```
alpha=0.7
```

✓ Cuadrícula

```
plt.grid(True, alpha=0.3)
```

Ejemplo completo: histograma personalizado

```
plt.figure(figsize=(8,5))
plt.hist(alturas, bins=10, color="skyblue", edgecolor="black", alpha=0.8)
plt.title("Distribución de alturas")
plt.xlabel("Altura")
plt.ylabel("Frecuencia")
plt.grid(axis="y", alpha=0.3)
plt.savefig("figures/hist_alturas.png")
plt.show()
```

ACTIVIDAD

Cada equipo debe:

1. Cargar sus datos limpios (data/clean/data_clean.json).

2. Elegir al menos 3 variables:

- 2 numéricas
- 1 categórica (si su API la tiene)

3. Generar al menos 4 gráficas: traten que sean de diferentes tipos, y si no, al menos, grafiquen diferentes datos (no solo valores).

4. Personalizar cada gráfica con:

- título
- etiquetas de ejes
- colores
- cuadrícula (si aplica)
- tamaño de figura
- guardado en carpeta figures/

5. Crear visualizations.py con funciones como:

- graficar_histograma(lista)
- graficar_barras(categorias, valores)
- graficar_linea(x, y)
- graficar_pastel(labels, valores)

6. Subir todo a GitHub.

TAREA

Cada equipo debe:

- Terminar visualizations.py.
- Subir todas las gráficas a GitHub.
- Revisar si sus gráficas tienen errores de escala o interpretación.

RESULTADO ESPERADO

Al finalizar el día, cada equipo debe tener:

- 4 gráficas correctas, claras y no distorsionadas
- Carpeta figures/ con imágenes guardadas
- Archivo visualizations.py funcional
- Avance directo hacia el Script 2

⚠ ORGANIZACIÓN OFICIAL DEL PROYECTO (Script 1 y Script 2)

A partir de este punto del PIA, **el proyecto debe estar dividido en dos grandes scripts**, cada uno compuesto por varios módulos.

Esto permite que el código sea profesional, mantenible y fácil de evaluar.

SCRIPT 1 — “Obtención, validación y limpieza de datos”

Este script se encarga de todo lo relacionado con traer datos desde la API y dejarlos limpios y listos para análisis.

El **Script 1** NO es un solo archivo.

Es un **conjunto de módulos** que trabajan juntos.

✓ Archivos que forman el Script 1

```
src/
├── api_client.py      → conexión a la API, manejo de errores, GET
├── validators.py      → validación con regex, validación de campos
├── cleaner.py         → limpieza, normalización, conversión de tipos
├── utils.py           → funciones auxiliares (opcional)
└── script1.py         → orquestador del Script 1
```

✓ ¿Qué hace cada módulo?

1. api_client.py

- Hace la conexión a la API
- Maneja errores (try/except)
- Revisa status_code
- Devuelve el JSON crudo

2. validators.py

- Contiene **todas** las expresiones regulares
- Valida campos numéricos, fechas, textos, códigos
- Imprime advertencias si algo no cumple

3. cleaner.py

- Convierte tipos (str → int, float)
- Aplana estructuras anidadas
- Normaliza nombres
- Maneja valores nulos
- Construye el diccionario limpio final

4. utils.py (opcional)

- Funciones auxiliares
- Impresión de banners
- Manejo de rutas

- Cualquier cosa que no pertenezca a los otros módulos

5. script1.py

- Es el **orquestador**
- Llama a api_client → validators → cleaner
- Guarda los datos limpios en:
- data/clean/data_clean.json

SCRIPT 2 — “Análisis, visualización y narrativa”

Este script toma los datos limpios del Script 1 y genera estadísticas, gráficas y hallazgos.

El **Script 2** también es un conjunto de módulos.

✓ Archivos que forman el Script 2

```
src/
├── analysis.py           → estadísticas descriptivas
├── visualizations.py    → gráficas (histograma, barras, líneas, pastel)
├── utils.py             → funciones auxiliares (compartido con Script 1)
└── script2.py           → orquestador del Script 2
```

✓ ¿Qué hace cada módulo?

1. analysis.py

- Calcula media, mediana, moda, rango, desviación estándar
- Lee el archivo limpio del Script 1
- Devuelve resultados listos para guardar

2. visualizations.py

- Genera todas las gráficas:
 - histograma
 - barras
 - líneas
 - dispersión
 - pastel
- Personaliza colores, títulos, etiquetas
- Guarda las gráficas en:
- results/figures/

3. utils.py

- Funciones auxiliares compartidas
- Manejo de rutas
- Creación de carpetas
- Impresión de mensajes

4. script2.py

- Es el **orquestador**
- Carga datos limpios
- Llama a analysis.py
- Llama a visualizations.py
- Guarda estadísticas en:

- results/stats/estadisticas.json
- Puede imprimir o guardar narrativa inicial

Estructura completa del proyecto (a estas alturas del PIA)

```
proyecto/
├── README.md
├── main.py (opcional)
├── src/
│   ├── api_client.py
│   ├── validators.py
│   ├── cleaner.py
│   ├── analysis.py
│   ├── visualizations.py
│   ├── utils.py
│   ├── script1.py
│   └── script2.py
├── data/
│   ├── raw/
│   └── clean/
└── results/
    ├── stats/
    └── figures/
```

Aclaración importante sobre parámetros (sys.argv)

- **Si su API requiere parámetros** (ciudad, fecha, ID, tipo, etc.), entonces deben implementarlos **desde Script 1**.
- **Si su API NO requiere parámetros**, pueden implementarlos **hasta Script 2** (entregables 3 y 4).

Esto depende totalmente del endpoint que eligieron.